

Guida Completa: Programmazione Dinamica e Algoritmi Greedy del Corso

PARTE I - PROGRAMMAZIONE DINAMICA

Schema Generale della DP nel Vostro Corso

Ricetta Standard per ogni Problema DP

1. **Caratterizzare la struttura** di una soluzione ottima s^* in funzione di soluzioni ottime $s_1^*, \dots, s_k^*$ di sottoistanze di taglia inferiore
2. **Determinare relazione di ricorrenza** del tipo $c(s^*) = f(c(s_1^*), \dots, c(s_k^*))$
3. *Calcolare $C(S)^*$ utilizzando la ricorrenza in modalità bottom-up (iterativo) oppure memoizzando il codice D&C*
4. **Mantenere informazioni strutturali** aggiuntive per ricostruire S^* (opzionale)

Pattern di Scansione delle Tabelle

Tutti gli algoritmi DP del corso seguono pattern di scansione specifici:

- **Row Major**: Per righe, da sinistra a destra
- **Column Major**: Per colonne, dall'alto al basso
- **Reverse Column Major**: Colonne decrescenti, righe crescenti
- **Reverse Row Major**: Righe decrescenti, colonne decrescenti
- **Diagonal**: Per diagonal parallele alla principale

Regola Fondamentale: I costi sono sempre dati da una percorrenza inversa della matrice rispetto al senso di andata.

Casi Base Standard per Ricorrenze DP

- $(i=0, j=0)$: Il costo è sempre 0
- Indice oltrepassa grandezza input: Si pone a $+\infty$
- Indice arriva a taglia input $(n-1)$ o 0: È una costante
- $(i, j > 0)$ ma $(X[i] = X[j])$: Si ha min/max degli indici calcolati
- $(i, j > 0)$ ma $(X[i] \neq X[j])$: Si ha min/max + soluzione ottima calcolata

Tipologie Complete di Esercizi DP del Corso

1. Problemi su Stringhe - LCS (Longest Common Subsequence)

Sottostruttura Ottima: $Z = \text{LCS}(X_i, Y_j)$ dove:

- Se $x_i = y_j$: Z termina con $x_i = y_j$ e $Z' = \text{LCS}(X_{i-1}, Y_{j-1})$
- Se $x_i \neq y_j$: Z è la più lunga tra $\text{LCS}(X', Y)$ e $\text{LCS}(X, Y')$

Ricorrenza sui Costi:

$$L[i, j] = |\text{LCS}(X_i, Y_j)|$$

$$L[i, j] = \begin{cases} 0 & \text{se } i=0 \text{ o } j=0 \\ L[i-1, j-1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j \\ \max(L[i-1, j], L[i, j-1]) & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Algoritmo Bottom-Up:

python

```
LCS_LENGTH(X, Y):
    m = length(X), n = length(Y)
    for i = 0 to m: L[i, 0] = 0
    for j = 0 to n: L[0, j] = 0

    for i = 1 to m:
        for j = 1 to n:
            if X[i] == Y[j]:
                L[i, j] = L[i-1, j-1] + 1
            else:
                L[i, j] = max(L[i-1, j], L[i, j-1])

    return L[m, n]
```

Calcolo Esatto Costi:

- Inizializzazione: $2n$ operazioni
- Ciclo principale: mn confronti + mn operazioni di assegnamento
- **Costo Totale:** $T(n) = 2n + 2mn = O(mn)$

Algoritmo Memoizzato (più efficiente in alcuni casi):

python

```
LCS_MEMO(X, Y, i, j, memo):  
    if memo[i,j] != -1: return memo[i,j]  
  
    if i == 0 or j == 0:  
        memo[i,j] = 0  
    elif X[i] == Y[j]:  
        memo[i,j] = 1 + LCS_MEMO(X, Y, i-1, j-1, memo)  
    else:  
        memo[i,j] = max(LCS_MEMO(X, Y, i-1, j, memo),  
                        LCS_MEMO(X, Y, i, j-1, memo))  
  
    return memo[i,j]
```

2. Problemi su Stringhe - LIS (Longest Increasing Subsequence)

Strengthening del Problema: Calcoliamo $LIS(X_i)$ = più lunga IS di X_i che termina proprio con X_i

Ricorrenza sui Costi:

$L[i]$ = lunghezza della LIS di $X[1..i]$ che termina con $X[i]$

$L[i] = 1 + \max\{L[j] : j < i \text{ e } X[j] < X[i]\}$
 $= 1$ se non esiste tale j

Algoritmo Bottom-Up:

python

```
LIS_LENGTH(X):  
    n = length(X)  
    for i = 1 to n: L[i] = 1, prev[i] = NULL  
  
    for i = 2 to n:  
        for j = 1 to i-1:  
            if X[j] < X[i] and L[j] + 1 > L[i]:  
                L[i] = L[j] + 1  
                prev[i] = j  
  
    return max(L[1..n])
```

Calcolo Esatto Costi: $T(n) = \sum_{i=2}^n \sum_{j=1}^{i-1} 1 = \sum_{i=2}^n (i-1) = (n-1)n/2 = O(n^2)$

3. Edit Distance

Ricorrenza sui Costi:

$e(i,j)$ = edit distance tra $X[1..i]$ e $Y[1..j]$

```
e(i,j) = {
    j                se i=0
    i                se j=0
    e(i-1,j-1)       se i,j>0 e xi=yj
    1 + min(e(i-1,j), e(i,j-1), e(i-1,j-1)) altrimenti
}
```

4. Problemi su Matrici - Catena di Moltiplicazioni

Esempio Tipo dal Vostro Corso: Ricorrenza $c(i,j)$ definita per $1 \leq i,j \leq n$:

```
c(i,j) = {
    aj                se i=1, 1≤j≤n
    an+1-i            se j=n, 1<i≤n
    c(i-1,j) · c(i,j+1) · c(i-1,j+1) altrimenti
}
```

Algoritmo con Scansione Reverse Column-Major:

python

```
COMPUTE_C(A):
    n = length(A)
    # Inizializzazione casi base
    for i = 1 to n:
        c[1,i] = a[i]
        c[i,n] = a[n+1-i]

    # Scansione reverse column-major
    for j = n-1 downto 1:
        for i = 2 to n:
            c[i,j] = c[i-1,j] * c[i,j+1] * c[i-1,j+1]

    return c[n,1]
```

Calcolo Esatto Moltiplicazioni: $T(n) = \sum_{j=1}^{n-1} \sum_{i=2}^n 2 = \sum_{j=1}^{n-1} 2(n-1) = 2(n-1)^2$

5. Shortest Path su Griglia

Problema Tipo: Cammino minimo da (1,1) a (n,n) con costi $u[i,j]$ (up) e $r[i,j]$ (right).

Ricorrenza:

$C[i,j]$ = costo minimo da (i,j) a (n,n)

```
C[i,j] = {
    0                                se i=n e j=n
    C[i+1,j] + u[i,j]              se j=n, i<n
    C[i,j+1] + r[i,j]              se i=n, j<n
    min(C[i+1,j] + u[i,j],         altrimenti
        C[i,j+1] + r[i,j])
}
```

Algoritmo con Scansione Reverse Row-Major:

python

```
MIN_PATH(u, r, n):
    C[n,n] = 0
    # Inizializzazione bordi
    for j = n-1 downto 1: C[n,j] = C[n,j+1] + r[n,j]
    for i = n-1 downto 1: C[i,n] = C[i+1,n] + u[i,n]

    # Riempimento tabella
    for i = n-1 downto 1:
        for j = n-1 downto 1:
            C[i,j] = min(C[i+1,j] + u[i,j], C[i,j+1] + r[i,j])

    return C[1,1]
```

6. Problemi con Massimo/Minimo

Esempio: Calcolare $M = \max\{c(i,j) : 0 \leq i \leq j \leq n-1\}$ dove:

```
c(i,j) = {
    ai                se 0<i≤n-1 e j=n-1
    bj                se i=0 e 0≤j≤n-1
    c(i-1,j) · c(i,j+1) se 0<i≤j<n-1
}
```

Pattern di Risoluzione DP

Identificazione Dipendenze

Per ogni ricorrenza, identificare le dipendenze tra indici per determinare l'ordine di scansione corretto.

Scelta della Scansione

- **Reverse Column-Major:** quando dipende da $(i-1,j)$, $(i,j+1)$, $(i-1,j+1)$
- **Reverse Row-Major:** quando dipende da $(i+1,j)$, $(i,j+1)$
- **Diagonal:** quando dipende da celle su diagonali precedenti

Calcolo Esatto dei Costi

Sempre richiesto negli esami. Per doppi cicli annidati:

```
 $\sum_{i=a}^b \sum_{j=f(i)}^{g(i)} \text{costo\_operazione}$ 
```

PARTE II - ALGORITMI GREEDY

Template di Base: Selezione Attività

Problema: Date n attività con tempi $[s_i, f_i]$, selezionare massimo numero di attività compatibili.

Strategia Greedy: Scegliere sempre l'attività con tempo di fine minore.

Algoritmo Template:

```
python

GREEDY_ACTIVITY_SELECTOR(s, f):
    n = length(s)
    A = {a1}  # Prima attività (tempo fine minore)
    last = 1

    for i = 2 to n:
        if s[i] >= f[last]:
            A = A U {ai}
            last = i

    return A
```

Dimostrazione Proprietà Greedy:

1. **Cut & Paste:** Ogni soluzione ottima può essere trasformata per contenere la scelta greedy
2. **Sottostruttura Ottima:** Il problema residuo mantiene la struttura ottima

Complessità: $O(n)$ se già ordinato, $O(n \log n)$ con ordinamento

Tipologie Complete di Greedy del Vostro Corso

1. Scheduling di Programmi

Problema: n programmi con lunghezze l_j , minimizzare $\sum_{j=1}^n C_j(\sigma)$ dove $C_j(\sigma)$ è tempo di completamento.

Strategia Greedy: Ordinare per lunghezza crescente.

Dimostrazione: Se abbiamo σ^* ottima con programma di lunghezza minima non primo, possiamo scambiare ottenendo σ' con costo $\leq \text{costo}(\sigma^*)$.

2. Metric Matching on the Line

Problema: Assegnare n client a n server minimizzando $\sum_i |c_i - s_j|$.

Strategia Greedy: Assegnare primo client (sinistro) al primo server (sinistro).

Algoritmo:

python

```
METRIC_MATCHING(S, C):
    n = C.length
    A = [C[1] - S[1]] # Prima assegnazione
    last = 1

    for i = 2 to n:
        for j = 1 to n-1:
            if C[i] - S[j] > C[last] - S[last] + 1:
                last = i
                A = A U [C[i] - S[j]]

    return A
```

Dimostrazione: Qualsiasi "incrocio" nelle assegnazioni aumenta il costo totale.

3. Copertura di Punti con Intervalli

Problema: Dato insieme X di punti sulla retta, coprire tutti con minimo numero di intervalli di lunghezza unitaria.

Strategia Greedy: Porre intervallo $[x_1, x_1+1]$ dove x_1 è il punto più a sinistra non coperto.

Algoritmo:

python

```
INTERVAL_COVER(X):
    Ordina X in ordine crescente
    I = ∅

    while X ≠ ∅:
        x1 = min(X)
        I = I U {[x1, x1+1]}
        X = X \ {x ∈ X : x ≤ x1+1}

    return I
```

Dimostrazione Cut & Paste: Se soluzione ottima I^* contiene $[a, a+1]$ con $a < x_1$, possiamo sostituire con $[x_1, x_1+1]$ ottenendo soluzione equivalente.

4. Esempi di Greedy Non Ottimi

Attività per Durata Minima:

python

```
NON_OTTIMO_ATTIVITA(s, f):  
    while esistono attività disponibili:  
        scegli attività i con durata minima ( $f_i - s_i$ )  
         $A = A \cup \{a_i\}$   
        rimuovi attività incompatibili con i  
    return A
```

Controesempio: $a_1(0,2)$, $a_2(1,3)$, $a_3(2,4)$, $a_4(0,10)$

- Algoritmo sceglie $\{a_1, a_2, a_3\}$
- Ottimo è $\{a_4\}$

5. Problemi con Monete

Problema: Dato valore n , selezionare minimo numero di monete con valori 50, 20, 1.

Algoritmo Greedy:

python

```
GREEDY_MONETE(n):  
    monete = [50, 20, 1]  
    soluzione = []  
  
    for valore in monete:  
        while n >= valore:  
            soluzione.append(valore)  
            n -= valore  
  
    return soluzione
```

Dimostrazione: Per $n = 60$, greedy restituisce 11 monete (non ottimo). Soluzione ottima: 3 monete da 20.

Dimostrazione Standard per Greedy

Schema Cut & Paste

1. **Supporre** esistenza soluzione ottima I^* che non contiene scelta greedy
2. **Costruire** soluzione I' sostituendo elemento di I^* con scelta greedy
3. **Dimostrare** che $|I'| \leq |I^*|$ e I' ammissibile
4. **Concludere** che I' è ottima e contiene scelta greedy

Sottostruttura Ottima

Dimostrare che rimosso la scelta greedy, il problema residuo mantiene struttura ottima.

PARTE III - CONFRONTO E DECISIONE

Quando Usare DP

- Sottoproblemi sovrapposti
- Sottostruttura ottima
- Scelte locali ottime non garantiscono optimum globale
- Problema di ottimizzazione

Quando Usare Greedy

- Proprietà di scelta greedy dimostrabile
- Sottostruttura ottima
- Scelte locali ottime conducono all'optimum globale
- Efficienza prioritaria

Strategia di Risoluzione Esami

Per DP

1. **Identificare** sottostruttura ottima
2. **Scrivere** ricorrenza sui costi
3. **Determinare** ordine di scansione dalle dipendenze
4. **Implementare** algoritmo bottom-up
5. **Calcolare** costo esatto con sommatorie

Per Greedy

1. **Proporre** strategia greedy
2. **Scrivere** algoritmo
3. **Dimostrare** proprietà greedy con cut & paste
4. **Dimostrare** sottostruttura ottima
5. **Calcolare** complessità

Tipici Errori da Evitare

- **DP**: Scansione errata della tabella, calcolo sbagliato dei costi
- **Greedy**: Dimostrazione incompleta, controesempi ignorati
- **Entrambi**: Sottostruttura ottima non verificata

Questa guida copre tutti i pattern e tipologie di esercizi DP e Greedy del vostro corso, con particolare attenzione ai metodi di calcolo esatto dei costi e alle dimostrazioni richieste negli esami.